DT124A - Pulse Width Modulation

Introduction

Mandatory lab. No report required.

Purpose

The purpose of this lab is:

- Implement PWM functionality and check limitations
- For a higher grade implement changing PWM

Equipment

- Laptop computer (students' own) with Microchip Studio IDE (or equivalent IDE)
- Arduino Uno (or similar microcontroller) and, possibly, AVR programmer
- Red LED
- Resistors (1 x 220Ω), and if needed others for LCD, see separate documentation.
- 16x2 LCD display.
- Potentiometer
- Separate sensor if you do not want to do task 2 with a potentiometer.

Preparation

In order to do this lab, you need to:

1. Read the section about theory.
2. Prepare the lab.

1 Grading

Note: To achieve grade 4 or 5, all previous grade requirements must also be met.

1.1 Grade 3 - Passing grade

Requirements:

- Implement and show task 1 with questions answered to instructor.
- Students must be ready to modify code and answer other questions about code.

1.2 Grade 4

Requirements:

- Fulfill requirements for grade 3.
- Implement and show task 2 and be ready to answer questions and defend your work.

1.3 Grade 5

Requirements:

- Fulfill requirements for grade 3 and grade 4.
- Implement and show task 3 and be ready to answer questions and defend your work.

2 Task 1 – Dimming LED using PWM

Write a program to dim a LED using a duty cycle of 80%, 50%, 40%, and 20%, using a timer configured to change the duty cycle every 1s.

Questions:

1. Which timer do you use?
2. What is the configuration of the timer?
   (a) Mode
   (b) TOP value
   (c) COMnX value
   (d) Prescaler
3. What pin did you connect the LED to?

The task has been completed successfully if the while loop is empty entirely and no delay function calls are in the program at all (Meaning you need to do it on interrupts), while the LED is dimmed.

3 Task 2 – PWM Mode Control using a Push Button

Modify the PWM program so that pressing a button cycles through different PWM duty cycles for the LED.

The system shall cycle through the following PWM duty cycles:

- 0%
- 50%
- 70%
- 100%

Each button press shall change to the next mode.

This type of functionality is commonly used in embedded systems such as:

- Small desk fans
- LED dimmers
- Power mode selectors
- Small appliances with multiple speed settings

Questions:

1. How is button input detected?
2. How do you keep track of the current PWM mode?
3. How is button debouncing handled?
4. Does the PWM continue running while waiting for button input?

The task has been completed successfully if the button correctly cycles through all PWM levels, PWM generation continues uninterrupted, and no blocking delay functions are used.

4 Task 3 – USART Controlled PWM with Timeout

Extend the PWM system so that the LED brightness can be controlled through USART communication.

The program shall:

- Receive commands through USART
- Change PWM duty cycle dynamically
- Automatically turn off the LED if no serial command has been received for 5 seconds

Example mapping:

- '0' → 0%
- '1' → 25%
- '2' → 50%
- '3' → 75%
- '4' → 100%

Questions:

1. How is serial data received?
2. How is the timeout implemented?
3. What happens when new USART data is received?
4. Which timer functionality is used?
5. Is the system polling- or interrupt-driven?

The task has been completed successfully if PWM brightness changes correctly through USART communication, the LED automatically turns off after timeout, PWM generation continues correctly, and no blocking delay functions are used.